

공통 컴포넌트와 가이드로 팀의 기준을 만들어 온 6년차 웹 퍼블리셔

공통 컴포넌트와 가이드, 반응형 통합 레이아웃, 인터랙션·애니메이션, AI 워크플로우를 퍼블리싱 관점에서 연결해 왔습니다.

GitHub [CHANSIK-CHOI](#) | Blog [Velog](#)

TECH STACK

Next.js React TypeScript

Zustand React Hook Form

Supabase HTML SCSS

Framer Motion Claude Code

StoryBook Git

CONTENTS

목차

퍼블리싱 실무 경력과 대표 프로젝트의
적용 사례, 그리고 그 역량을 바탕으로
확장한 개인 프로젝트로 구성되어
있습니다.

1	경력 기술서 퍼블리싱 중심 실무 경험
2	롯데월드 차세대 프로젝트 개인 프로젝트 : Next.js 기반 공통 UI 컴포넌트 가이드
3	드시모네몰 리뉴얼 · 한화 ESG 프로젝트 개인 프로젝트 : Claude Code 기반 AI 협업형 Vue UI 컴포넌트 제작 시스템
4	프론트엔드 개발 경험 : 인터뷰어 피드백 보드 상태, 권한, 코멘트, 알림 흐름 설계

퍼블리싱 실무 경험

프로젝트	기간	기술	주요 작업
한화 ESG 대시보드 구축	2026.07 - 진행중	Vue, TypeScript, Storybook, Claude Code, 패키지 레지스트리	공통 컴포넌트 전체 제작 및 사내 레지스트리 배포 / Claude Code 스킬 설계로 컴포넌트 제작 1개월→2주 단축 / 기획 HTML→Vue 전환 스킬로 20개 페이지 제작
드시모네몰 리뉴얼	2026.04 - 2026.07	Nuxt, Vue, TypeScript, SCSS, Radix Vue, Vant, Claude Code	퍼블리싱 PL / 공통 컴포넌트 약 20종·반응형 레이아웃 제작 / 팝업 포함 147개 화면을 2명이 1개월 만에 완료
롯데월드 차세대	2024.10 - 2025.09	React, SCSS, Framer Motion, Swiper, react-select, react-day-picker	테크 리더 / 7개 사이트 공통 React 컴포넌트 약 25종·반응형 레이아웃 설계 / 인터랙션·가이드 구축 / 웹어워드 코리아 2025 대상
전북은행 쓱뱅크	2023.09 - 2024.02	HTML, SCSS, JavaScript, jQuery, React, Highcharts	앱 WebView UI 개발 / Highcharts 데이터 시각화 구현 / 차트 옵션·이벤트 공통화로 재사용 구조 정리 / 스마트앱어워드 2024 최우수상

퍼블리싱 실무 경험

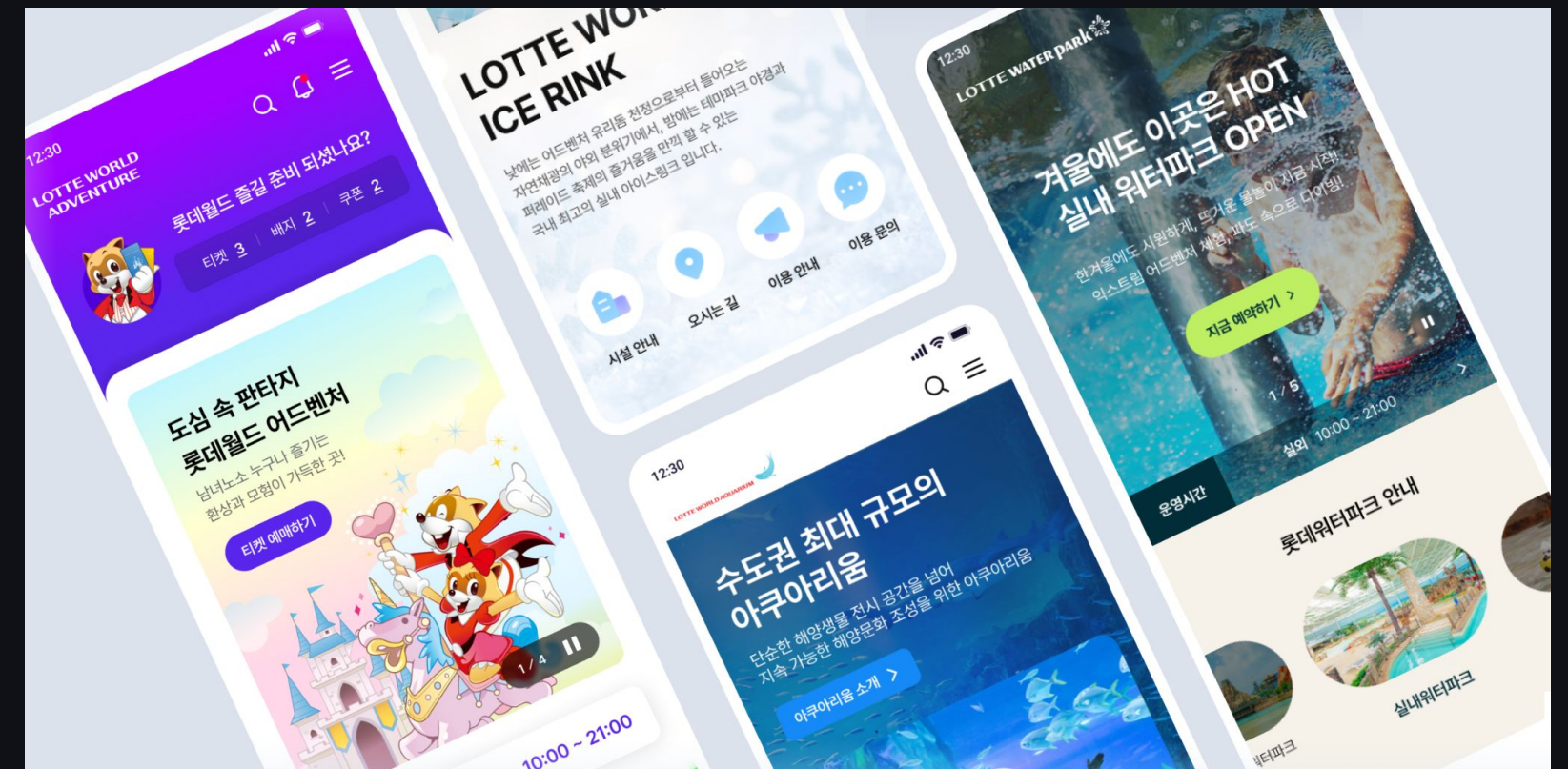
프로젝트	기간	기술	주요 작업
포스코 Flow	2023.04 – 2023.06	HTML, SCSS, jQuery, IBSheet8	대시보드 UI 개발 / IBSheet8 데이터 중심 UI·컴포넌트 가이드 구축 / 컬럼 유형·상태 표시를 공통 객체로 정의해 확장 구조 설계
국민은행 빌드·배포 자동화	2022.10 – 2023.03	React, TypeScript, SCSS, Ant Design, PrimeReact	대시보드 UI 개발 / Ant Design·PrimeReact 그리드를 Wrapper로 추상화해 단일 인터페이스 제공 / 케이스별 사용 기준 문서화
캐논코리아 통합	2022.03 – 2022.10	HTML, SCSS, jQuery	적응형 UI 개발 / 반응형 화면·공통 UI 컴포넌트 구현 / 사용 규칙 가이드화 / 웨어워드 코리아 2022 대상
하나손해보험 원데이 앱	2021.07 – 2022.03	HTML, CSS, JavaScript, GSAP	앱 UI 개발 / GSAP 인터랙션·애니메이션 구현 / 애니메이션 동작 기준 가이드화 / 스마트앱어워드 2021 최우수상

롯데월드 차세대 프로젝트

롯데월드 어드벤처·부산, 아쿠아리움, 워터파크, 아이스링크, 민속박물관, 예매·단체 예매 서비스 등 여러 사이트를 하나의 기준으로 묶는 통합 구축 프로젝트입니다.

저는 여러 사이트가 공통으로 사용하는 반응형 공통 레이아웃과 UI 컴포넌트, 인터랙션, 가이드를 직접 설계하고 구축했습니다.

- 기간 : 2024.10 ~ 2025.09
- 기술 : React, SCSS, Framer Motion, Swiper, react-select, react-day-picker
- 담당 : 테크 리더 / 공통 레이아웃·컴포넌트 약 25종·인터랙션·가이드 구축
- 🏆 2025 웹어워드 코리아 — 문화/레포츠분야 대상, 레저/스포츠분야 대상



롯데월드 차세대 프로젝트

핵심 포인트

1

모노레포 기반 통합 기준 설계

기능은 공통, 스타일은 사이트별로 책임을 분리하여 유지보수성 향상

2

앱 WebView & Web 반응형 UI 구현

크로스브라우징·웹 표준을 고려해 마크업

3

모든 공통 컴포넌트와 레이아웃, 퍼블리싱 구조 작업 100% 수행

문서화를 통해 퍼블리셔와 개발자간 같은 기준으로 작업할 수 있도록 유도

모노레포 기반 통합 기준 설계

하나의 저장소에서 **공통 모듈(shared)**과 각 사이트를 함께 관리하는 **모노레포 형태의 구조로 설계**했습니다.

공통 컴포넌트의 구조와 기능(script)은 **shared**에서 한 곳으로 관리해 여러 사이트가 같은 기준으로 동작하도록 하고, 사이트마다 달라지는 **디자인 토큰과 컴포넌트 스타일**은 각 사이트 폴더에서 관리했습니다.

이렇게 ‘기능은 공통, 스타일은 사이트별’로 책임을 분리해, **공통 기준을 일관되게 유지**하면서도 **사이트별 디자인 차이와 유지보수에 유연하게 대응**할 수 있는 구조를 만들었습니다.

```
lotteworld/
├── shared/                # 모든 사이트가 공유하는 공통 모듈
│   ├── components/      # 공통 UI 컴포넌트
│   ├── layouts/         # 공통 레이아웃
│   ├── styles/          # 공통 SCSS (mixin, reset)
│   └── hooks/           # 공통 훅
│
├── 어드벤처/
│   └── styles/           # 어드벤처 사이트의 컴포넌트 스타일
├── 어드벤처 부산/
├── 아쿠아리움/
├── 워터파크/
├── 아이스링크/
├── 민속박물관/
├── 예매 서비스/
└── 단체 예매 서비스/
```

앱 WebView & Web 반응형 UI 구현

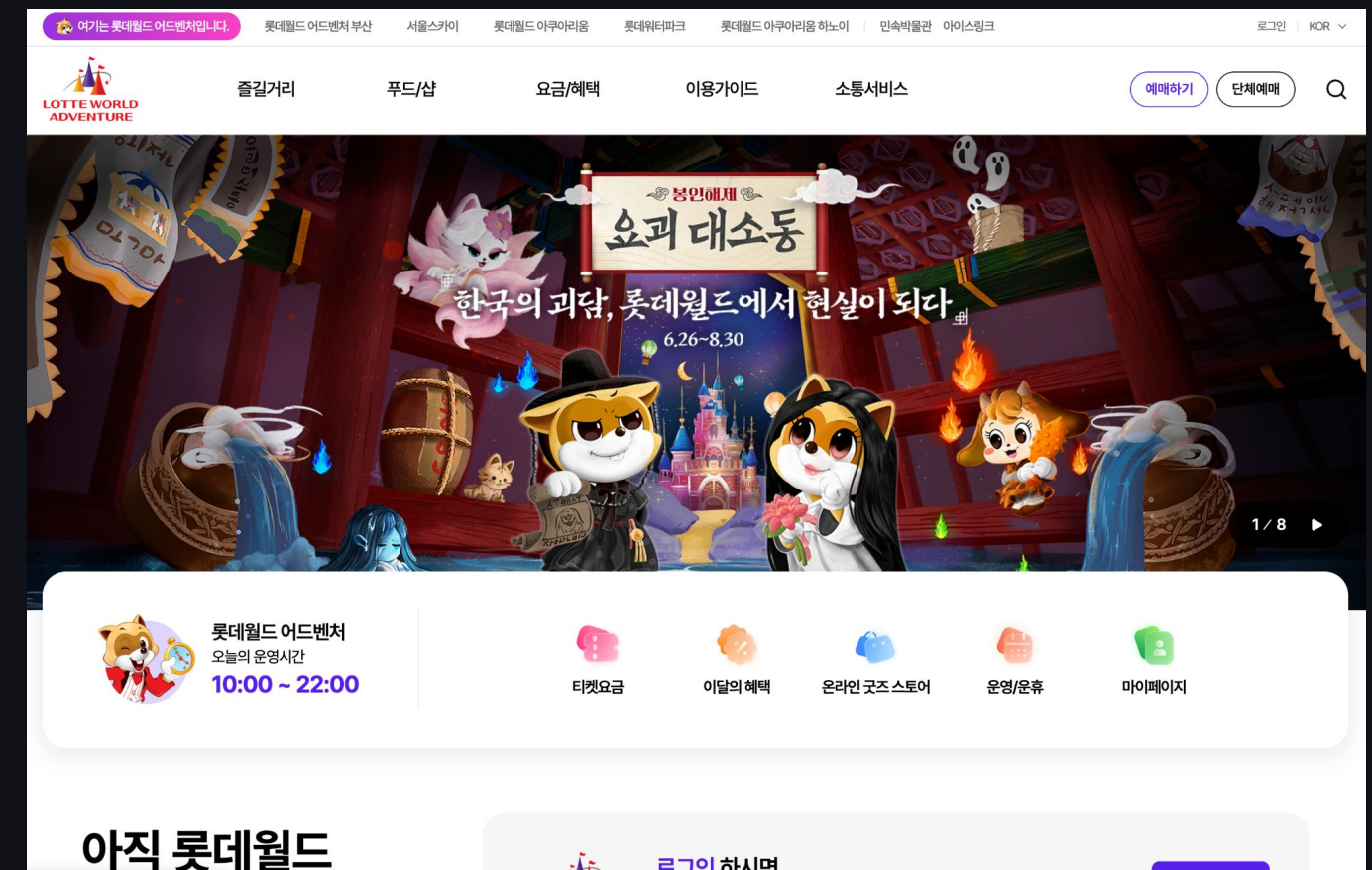
롯데월드 APP



[반응형] 롯데월드 Web Mobile



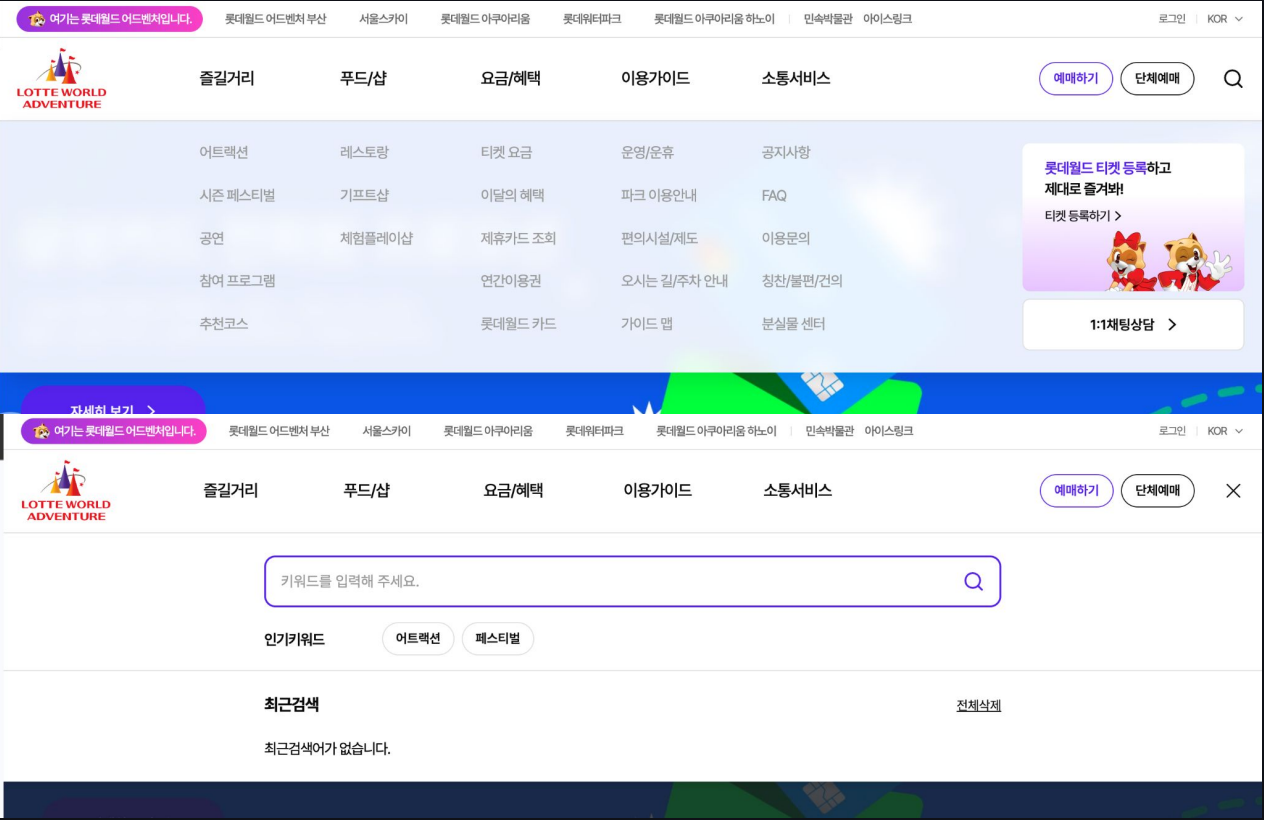
[반응형] 롯데월드 Web PC



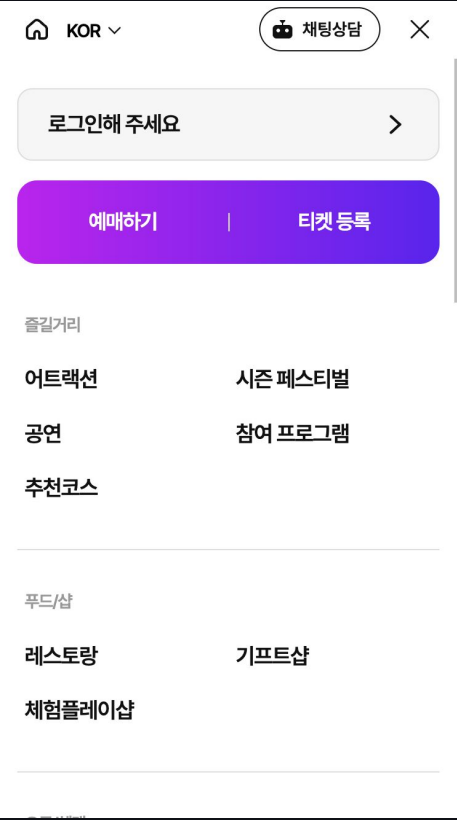
모든 사이트를 반응형으로 구축하고, 다양한 화면과 브라우저 환경에서 안정적으로 동작하도록 크로스브라우징·웹 표준을 고려해 마크업했습니다.

모든 공통 컴포넌트와 레이아웃, 퍼블리싱 구조 작업 100% 수행

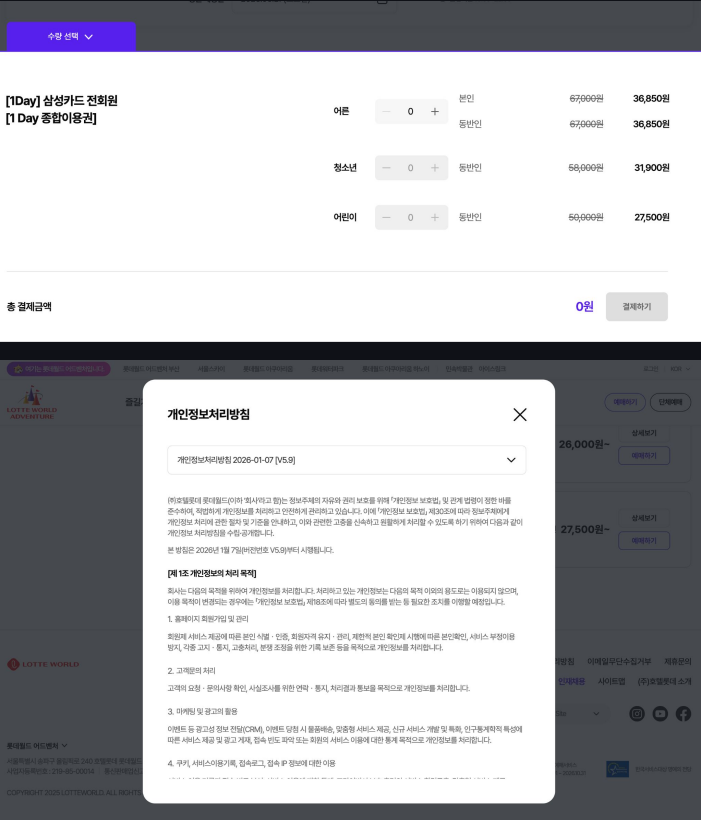
Header, Navigation, 검색 영역



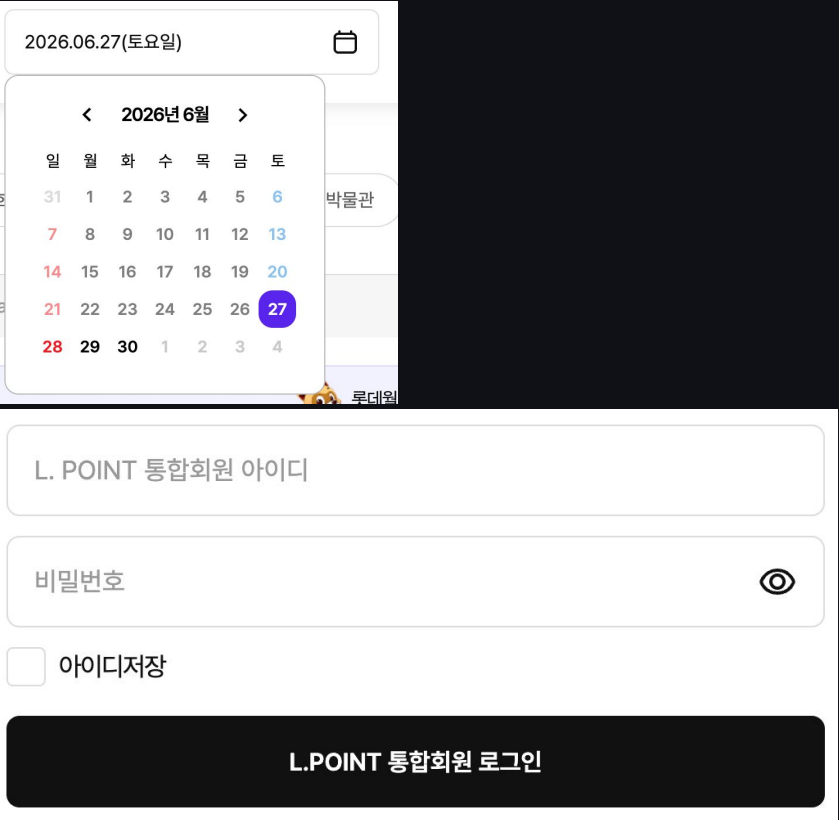
모바일 전체 메뉴



팝업



인풋 & 버튼 등 공통 컴포넌트



React 환경에서 헤더·푸터·네비게이션 등 모든 페이지가 공유하는 **반응형 공통 레이아웃**과 입력·팝업 등 **공통 컴포넌트**를 처음부터 끝까지 직접 구축했습니다. 인터랙션과 슬라이드가 필요한 컴포넌트는 **Framer Motion·Swiper**로 구현했고, 공통 컴포넌트의 **옵션과 사용 규칙**을 가이드로 남겨 여러 퍼블리셔·개발자가 같은 기준으로 작업하고 화면별 구현 편차를 줄일 수 있도록 했습니다.

롯데월드 프로젝트를 마친 뒤, 함께 일한 개발자들에게 받은 **피드백을**
통해 당시 컴포넌트 구조의 한계를 파악했습니다.
이를 더 나은 방향으로 개선하기 위해 **개인 프로젝트로 직접**
재설계했습니다.

마주한 문제와 해결을 위한 핵심 설계 판단

롯데월드 프로젝트에서 마주한 문제

- 1 입력 컴포넌트가 내부 state 중심으로 설계되어 외부 제어가 어려운 문제
- 2 React Hook Form 과 같은 폼 라이브러리와 자연스럽게 연결되지 않는 문제
- 3 Popup / Toast 등 전역 UI 상태가 여러 Context로 분산되어 흐름 추적이 어려운 문제

핵심 설계 판단 & 개선 방향 설계

- 1 입력 컴포넌트는 내부 상태가 아니라 외부에서 제어하는 controlled 구조로 전환
- 2 UI 표현과 폼 상태 관리를 분리하기 위해 React Hook Form Wrapper 도입
- 3 전역 UI 상태는 분산된 Context가 아닌 단일 store(Zustand)로 통합
- 4 Base 컴포넌트와 Wrapper 컴포넌트를 분리하여 확장 구조 설계

Next.js 기반 공통 UI 컴포넌트 가이드

퍼블리싱 관점에서 출발한 프로젝트 — 롯데월드에서 공통 UI를 만들며 마주한 유지보수 문제를, 퍼블리셔로서 다시 분석해 컴포넌트 구조 설계까지 직접 검증했습니다.

controlled 컴포넌트와 React Hook Form Wrapper를 분리하고 Popup/Toast/Datepicker 같은 복합 UI 상태를 일관된 구조로 관리하였습니다.

프로젝트 보러가기

배포사이트
[Next UI Components Guide](#)

GitHub
[레파지토리](#)

Next.js

React Hook Form

Zustand

TypeScript

Framer Motion

React-Select

React-Day-Picker

이번 프로젝트에서 유지한 것 : 확장 가능한 Base & Wrapper 아키텍처 설계

입력 컴포넌트

Search Password Datepicker Rangepicker

Textfield

입력 컴포넌트의 공통 요소 및 기능 담당

- input 영역 + 텍스트 삭제 버튼
- native input props + TextfieldProps
- input 하단 메시지 영역

각 입력 컴포넌트 별 기능 확장

Search : Textfield + 검색 버튼 & TextfieldProps + SearchProps

Password : Textfield + 비밀번호 노출/비노출 버튼 & TextfieldProps + PasswordProps

팝업 컴포넌트

Alert Confirm BottomSheet FullPopup

PopupBase

팝업 컴포넌트의 공통 요소 및 기능 담당

- Header, Body, Footer, CloseBtn ...
- onOk, onCancel, onOpenComplete ...
- PopupBaseProps

각 팝업 컴포넌트 별 필요한 형태 및 기능 맞춤

Alert : Header Title, Body Text, Ok 버튼 & Alert에 필요한 이벤트

BottomSheet : type="bottomsheet" 로 형태와 인터렉션 변경 & BottomSheet에 필요한 이벤트

공통 형태, 인터랙션, Props/Event는 **Base 컴포넌트**가 담당하고,
이를 기반으로 **추가 기능과 UI 확장은 Wrapper 컴포넌트**에서 구현해 로직을 분리했습니다.

“

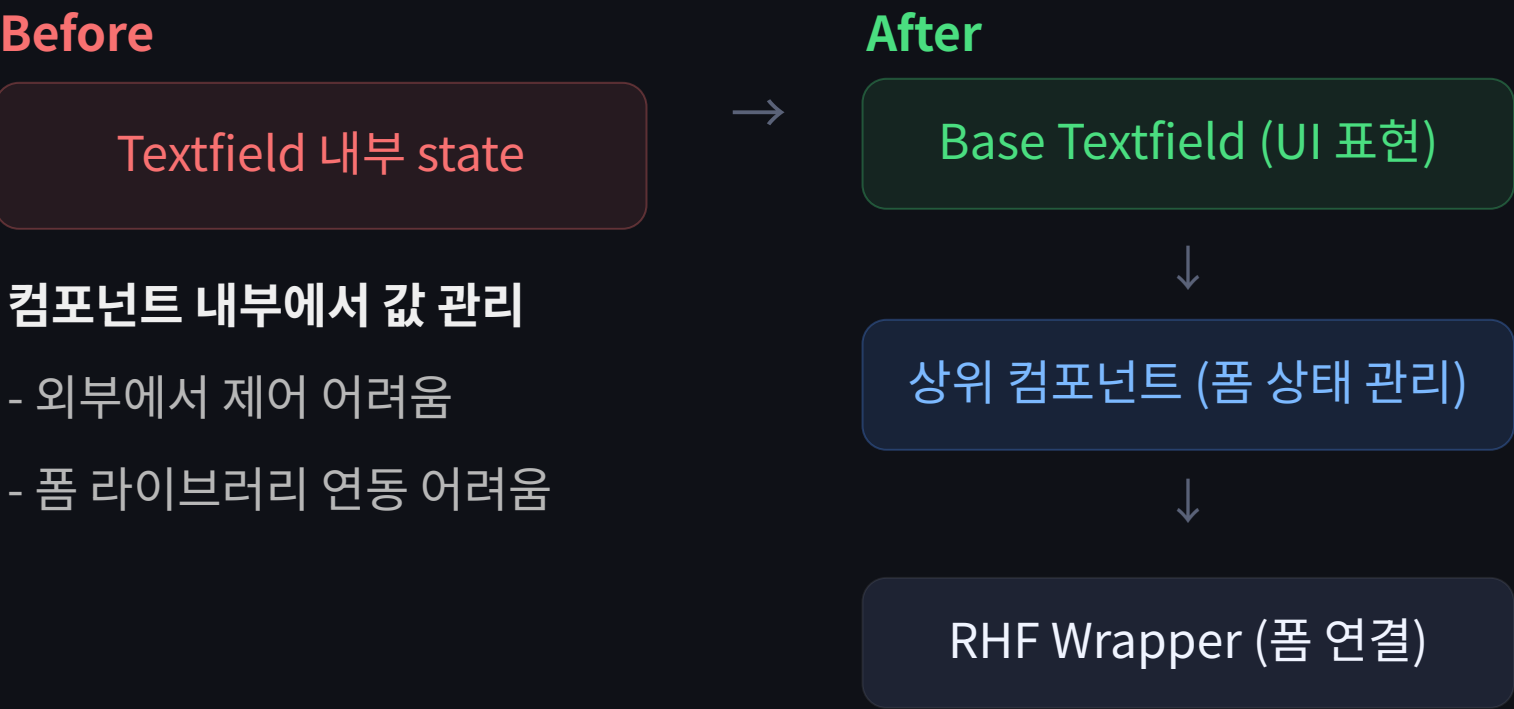
컴포넌트 내부 로직을 기능 단위로 분리해 관리함으로써, 기본 구조의 안정성을 유지하면서도 다양한 타입의 컴포넌트 확장에 유연하게 대응했습니다.

이 방식은 롯데월드 프로젝트에서 사용한 구조를 바탕으로, 개인 프로젝트에서 개선해 적용했습니다. 이후 개인 프로젝트에서도 동일한 설계 원칙으로 적용했습니다.

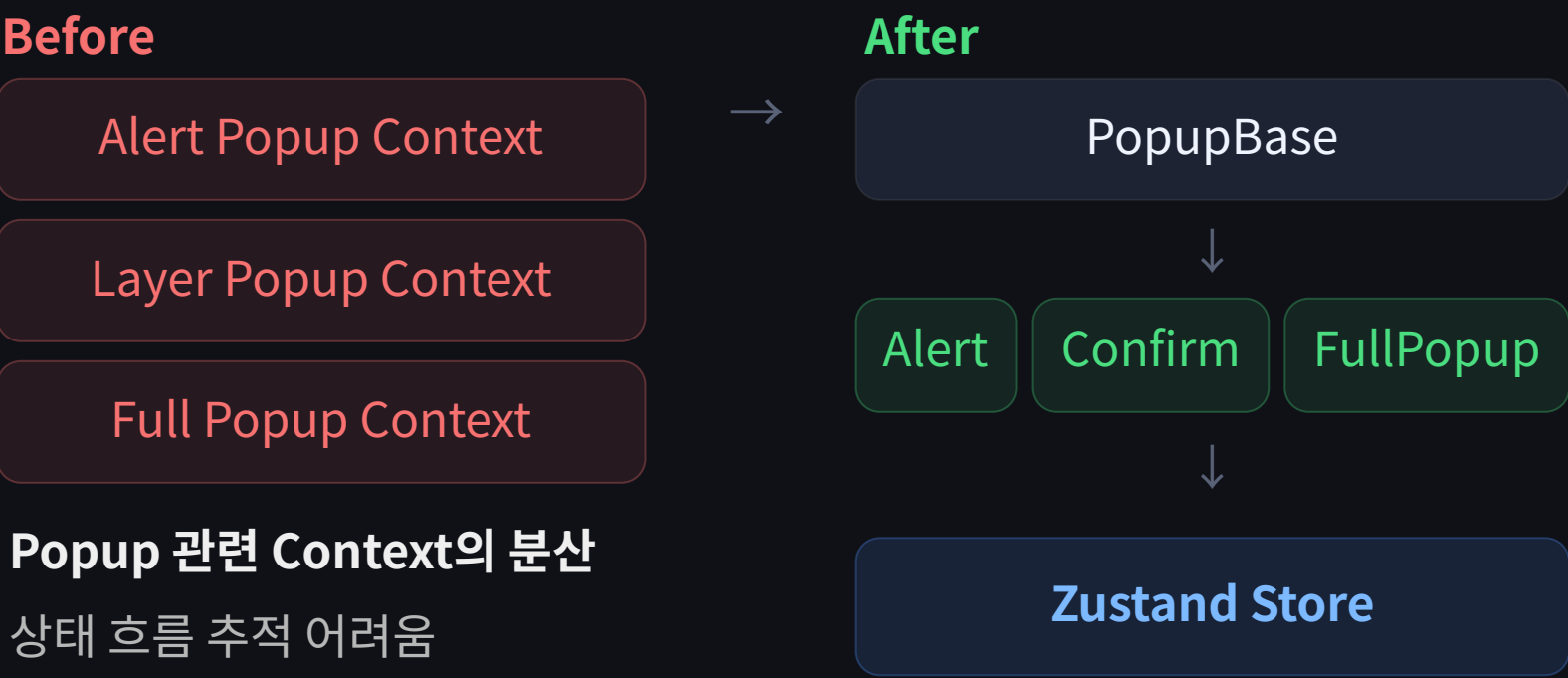
”

이번 프로젝트에서 개선한 것 : 입력 컴포넌트 구조 + Popup 상태 구조

입력 컴포넌트



Popup 상태 구조



- 입력 컴포넌트는 controlled 구조로 전환, 외부에서 제어
- UI 표현과 폼 상태 관리를 분리하기 위해 React Hook Form Wrapper 도입 (TextField + React Hook Form 전용 로직) => RHFTextField 컴포넌트

- 전역 UI 상태는 분산된 Context가 아닌 단일 store(Zustand)로 통합 (여러 형태의 팝업에 대한 로직을 store 파일 한 곳에서 관리) => [popup.store.ts](#) 확인

롯데월드 프로젝트 당시에는 개발 편의성을 고려한 구조화보다 기능 구현에 집중해,

UI와 비즈니스 로직을 컴포넌트 내부에서 함께 관리했습니다.

또한 ‘컴포넌트별 로직 분리’에만 집중한 나머지 Context까지 과도하게 분리했고,

그 결과 여러 형태의 팝업을 동시에 제어할 때 상태 흐름을 추적하기 어려운 문제가 발생했습니다.

실제로 실무 개발자 피드백을 통해 이 한계를 인식했고,

이번 개인 프로젝트에서는 해당 구조적 문제를 개선하는 데 집중했습니다.

“

개선한 컴포넌트 구조 역시 실제 프로젝트에 적용하는 과정에서 새로운 문제를 만날 수 있다고 생각합니다. 완벽한 설계는 없다는 전제 아래, 앞으로도 프로젝트를 진행하며 발생하는 이슈를 지속적으로 분석하고 개선해 나가겠습니다.

”

Before / After

	Before	After
입력 컴포넌트	내부 state 중심	controlled 구조
폼 연동	개별 구현 필요	RHF Wrapper로 통합
Popup 상태	Context 분산	단일 store

GitHub
[레파지토리](#)

배포 사이트
[Next UI Components Guide](#)

결과

- 입력 컴포넌트의 상태 관리 방식이 프로젝트 전반에서 일관되게 통일
- React Hook Form 연동 시 필요한 로직만 추가하여 사용 가능
- Popup 상태 흐름을 단일 구조로 통합하여 상태 흐름을 추적하기 쉬운 구조로 개선
- 새로운 UI 추가 시 기존 구조 수정 없이 확장 가능

인사이트

- 입력 컴포넌트의 핵심은 UI가 아니라 상태 제어 구조다
- 전역 상태는 분산보다 단일 기준이 유지보수에 유리하다
- 재사용성은 컴포넌트 형태가 아니라 상태 구조에서 결정된다

드시모네몰 리뉴얼 프로젝트

Claude Code 기반 퍼블리싱 워크플로우 설계

Claude Code를 단순 코드 생성 도구가 아니라, 제 작업 기준을 반영해 컴포넌트·페이지를 일관되게 제작하고 검수·테스트까지 연결하는 워크플로우로 활용했습니다.

- 기간 : 2026.04 ~ 2026.07
- 기술 : Nuxt, Vue, TypeScript, SCSS, Radix Vue, Vant, Claude Code



드시모네몰 리뉴얼 프로젝트

핵심 포인트

1

기준 정의 : 내 작업 규칙을 AI에 명문화

그동안 쌓아 온 아키텍처, 코드 컨벤션, 컴포넌트 작성 규칙을 Claude Code에 기준으로 정리해, AI가 제 기준으로 결과물을 만들도록 설계했습니다.

2

제작 : 동일한 기준으로 컴포넌트·페이지 생성

정의한 기준을 바탕으로 컴포넌트와 페이지를 일관된 방식으로 제작해, 사람·상황에 따라 달라지던 작업 편차를 줄였습니다.

3

검수·테스트 : 규칙 위반 자동 검출

검수와 테스트 단계를 워크플로우에 포함해, 정의한 규칙에 어긋나는 부분을 자동으로 잡아내도록 구성했습니다.

결과 : 생산성과 일관성 동시 확보 공통 컴포넌트 제작 기간을 1개월에서 2주로 단축했고, 팝업 포함 147개 화면을 2명이 1개월 만에 완료하며 결과물의 품질 편차를 통제했습니다.

한화 ESG 대시보드 구축 프로젝트

기획-디자인-퍼블-개발을 잇는 AI 파이프라인 설계

기획 1명 · 디자인 1명 · 퍼블리셔 1명 체제에서 퍼블리싱 영역의 컴포넌트 구성, 아키텍처, 자동화 스킵 설계를 단독으로 담당하고 있습니다.

- 기간 : 2026.07 ~ 진행중
- 기술 : Vue, TypeScript, Storybook, Claude Code, 사내 프라이빗 패키지 레지스트리



한화 ESG 대시보드 구축 프로젝트

핵심 포인트

1

컴포넌트 제작 스킬 설계

공통 컴포넌트 제작 기간을 1개월에서
2주로 단축.

2

기획 HTML → Vue 전환 스킬

디자인팀과 정의한 골격 템플릿 +
레이아웃 컴포넌트를 조합해 1개 대메뉴
20개 페이지 제작

3

파이프라인 확장

기획-디자인-퍼블 구간 연결 완료,
퍼블→개발 이관 스킬 설계 중

설계 의도

AI 도구의 핵심은 **‘얼마나 빠르게’가 아니라 ‘얼마나 일관되게, 검증 가능하게’** 만드느냐에 있다고 생각합니다.

Claude Code를 코드 생성기가 아니라 **제 퍼블리싱 기준을 일관되게 실행하는 도구**로 다뤘습니다.

이 방식을 더 체계화하여 개인 프로젝트를 진행했습니다.

Claude Code 기반 AI 협업형 Vue UI 컴포넌트 제작 시스템

Claude Code를 단순 코드 생성 도구로 사용하지 않고 **기획, 구현, QA, 리뷰 역할을 Agent 단위로 분리**해 반복적인 UI 컴포넌트 제작 과정을 단계와 검증 기준을 표준화한 프로젝트입니다.

Figma MCP, Context7 MCP, Playwright MCP를 활용해 디자인 토큰 추출, 라이브러리 API 확인, 화면 검증까지 **하나의 워크플로우로 연결**했습니다.

Claude Code

Vue

Figma MCP

Context7 MCP

Playwright MCP

핵심 설계 판단

- 1 컴포넌트 제작을 '코드 생성'이 아니라 '프로세스'로 정의
- 2 역할을 분리한 Agent 구조로 책임을 명확히 분할
- 3 생성 결과를 그대로 사용하지 않고 검증 단계를 강제

프로젝트 보러가기

배포사이트

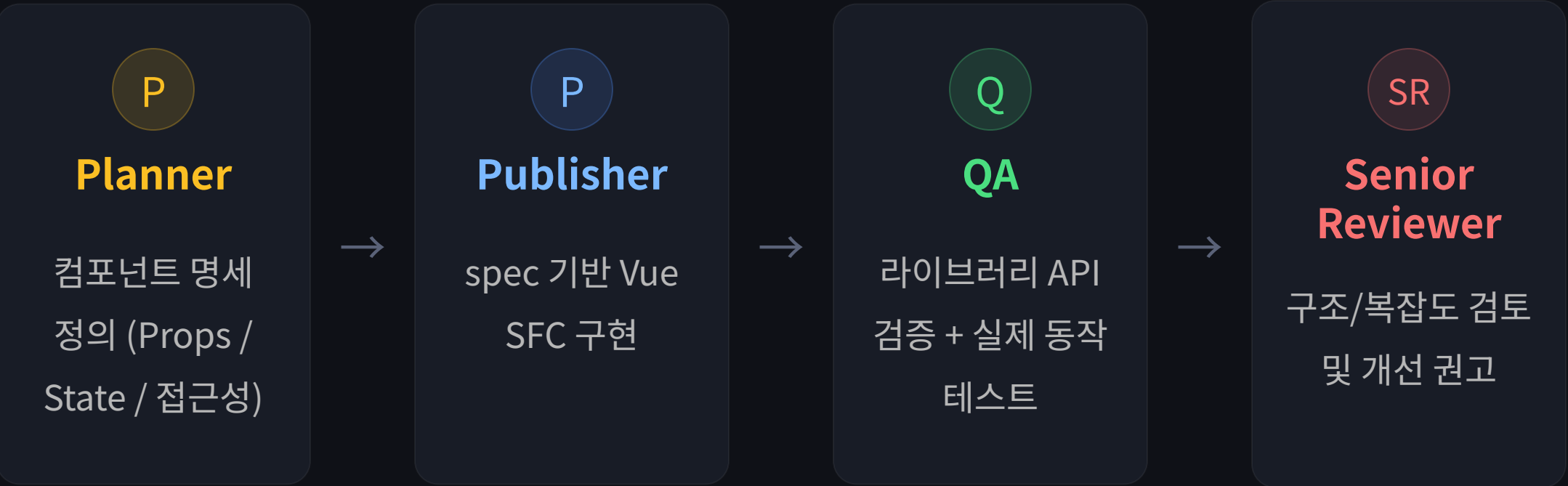
[Claude Code 기반 AI
협업형 Vue UI 컴포넌트](#)

GitHub

[레파지토리](#)

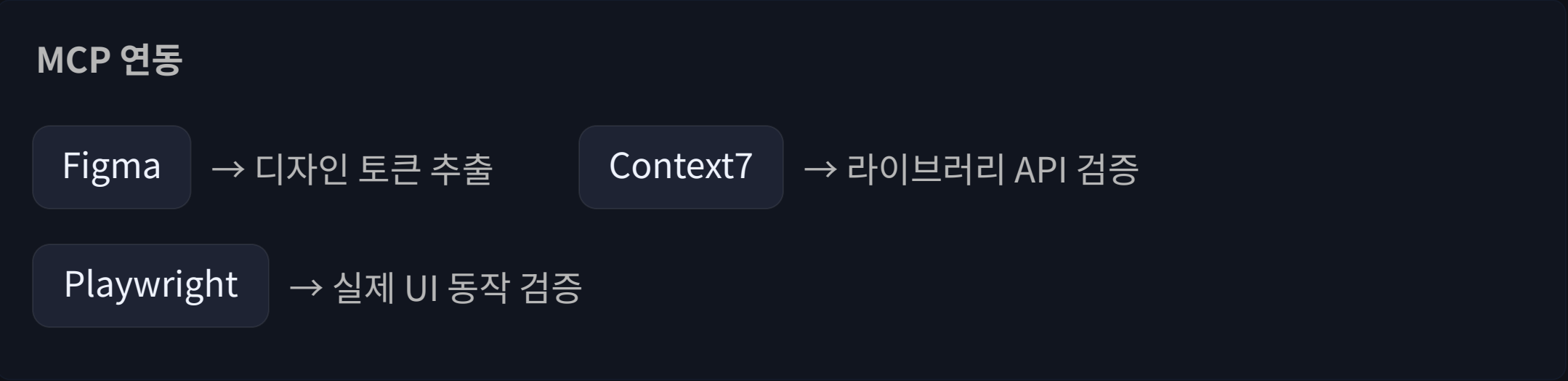
“
컴포넌트를 어떻게 만들어야 하는가 에 대한 제 철학을 Claude에 정리하고,
이를 기반으로 AI와 협업해 목표 결과물을 정밀하게 구현했습니다.
”

Agent 기반 역할 분리



설계 의도
각 단계가 명확한 입력과 출력(spec, code, report)을 가지도록 설계하여, AI가 임의 판단으로 흐름을 깨는 것을 방지했습니다.

Hallucination 방지
문서 기반 검증 + 실행 기반 검증



“ 각 에이전트는 역할 범위에 벗어난 파일을 수정하지 않고, 프로젝트의 각 파트처럼 자신의 책임에 충실히 동작하도록 설계했습니다. ”

워크플로우 구조



루프백 분기 정책

- 설계 자체 결함 : Planner부터 다시 실행
- 구현 영역 결함 : Publisher만 다시 실행
- 시니어 리뷰 영역 : 항상 Publisher만 다시 실행

최대 2회 루프

설계 의도

AI 결과를 단순 수용하지 않고, 검증 실패 시 이전 단계로 되돌리는 구조를 설계했습니다.

Before / After

	Before	After
컴포넌트 제작	개발자마다 방식 다름	표준화된 워크플로우
명세	암묵적	명시적 spec
검증	수동 리뷰	자동 + 검증 단계 강제
AI 활용	단순 코드 생성	프로세스 기반 활용
품질	편차 발생	기준 기반 일관성 확보

GitHub
[레파지토리](#)

배포 사이트
[Claude Code 기반 AI 협업형 Vue UI 컴포넌트](#)

결과

- 컴포넌트 제작 과정이 개인 의존이 아닌 프로세스 기반으로 전환
- 명세와 검증 기준이 명확해져 결과물 품질 편차를 통제할 검증 기준 마련
- 반복 작업을 명령어 기반으로 자동화하여 개발 흐름 단순화

인사이트

- AI는 코드를 생성하는 도구가 아니라, 기준을 수행하는 실행자에 가깝다
- 중요한 것은 "얼마나 빠르게"가 아니라 "얼마나 일관되게 만들 수 있는가"
- 개발 생산성은 코드가 아니라 프로세스에서 결정된다

**퍼블리셔로서의 전문성을 높이기 위해, 개인 프로젝트로 프론트엔드
개발을 직접 경험하며 역량을 넓히고 있습니다.**

**구현 과정을 이해할수록, 개발자와 더 정확하게 소통하고 협업할 수
있다고 생각합니다.**

인터뷰어 피드백 보드 - 프론트엔드

사용자가 작성한 피드백을 관리자가 검토 후 공개하고, 이후 댓글과 알림으로 상호작용이 이어지도록 설계한 권한 기반 검토/협업 워크플로우형 피드백 보드입니다.

Next.js Page Router와 Supabase를 기반으로 인증, 권한, 상태 전이, 댓글, 실시간 알림, API Routes를 구현했습니다.

단순 CRUD가 아니라 작성 → 검토 → 공개 → 댓글/답글 → 알림 → 확인으로 이어지는 사용자 행동 흐름을 기준으로 UI와 API 구조를 재설계했습니다.

또한 공개 목록과 사용자별 비공개 데이터를 분리하고, 각 상태에서 가능한 행동과 권한을 함께 정의해 기능 확장 시 충돌을 줄였습니다.

프로젝트 보러가기

배포사이트

[인터뷰어 피드백 보드 바로가기](#)

GitHub

[레파지토리](#)

- Next.js
- Supabase
- Page Router
- API Routes
- HttpOnly Cookie

“

테스트 계정

Reviewer : reviewer@gmail.com / Reviewer1!

Admin : admin@gmail.com / adminadmin1!

직접 회원 가입 후 사용해보셔도 됩니다.

코드 구조, API 설계, UX 개선 포인트에 대한 피드백을 편하게 남겨주시면 감사하겠습니다.

”

인터뷰어 피드백 보드

핵심 포인트

1

Next.js 14 Page Router,
TypeScript, Supabase 기반
구현

2

인증, 권한, 상태 전이, 코멘트,
알림, API Routes 구성

3

작성 → 검수 → 공개 →
코멘트/답글 → 알림 → 확인 흐름
설계

설계 포인트

이 프로젝트에서는 화면을 만드는 것보다, 권한과 상태가 바뀌어도 규칙이 흔들리지 않는 구조를 만드는 데 집중했습니다.

핵심 포인트

1

세션 동기화 : 브라우저 세션과 서버 HttpOnly 쿠키를 동기화해 클라이언트와 서버 기준을 맞췄습니다.

2

역할 분리 : reviewer와 admin의 데이터 가시성과 가능한 행동을 분리했습니다.

3

상태 전이 : pending, revised_pending, approved, rejected 상태를 API에서 강제했습니다.

4

상호작용 정책 : 승인 이후에만 코멘트 스레드가 열리도록 하고, 알림 흐름도 권한 기준에 맞춰 분리했습니다.

검증 포인트

구현에서 끝내지 않고, 주요 흐름이 실제로 유지되는지 확인할 수 있는 기준도 함께 두었습니다.

핵심 포인트

1

Playwright로 주요 E2E 시나리오를 설계하고, AI를 테스트 케이스 보완과 리뷰에 활용했습니다.

2

알림이 화면에서 올바른 문구와 상태 색상으로 표시되도록, 데이터 변환 로직과 유형 별 표시 규칙을 단위 테스트로 검증했습니다.

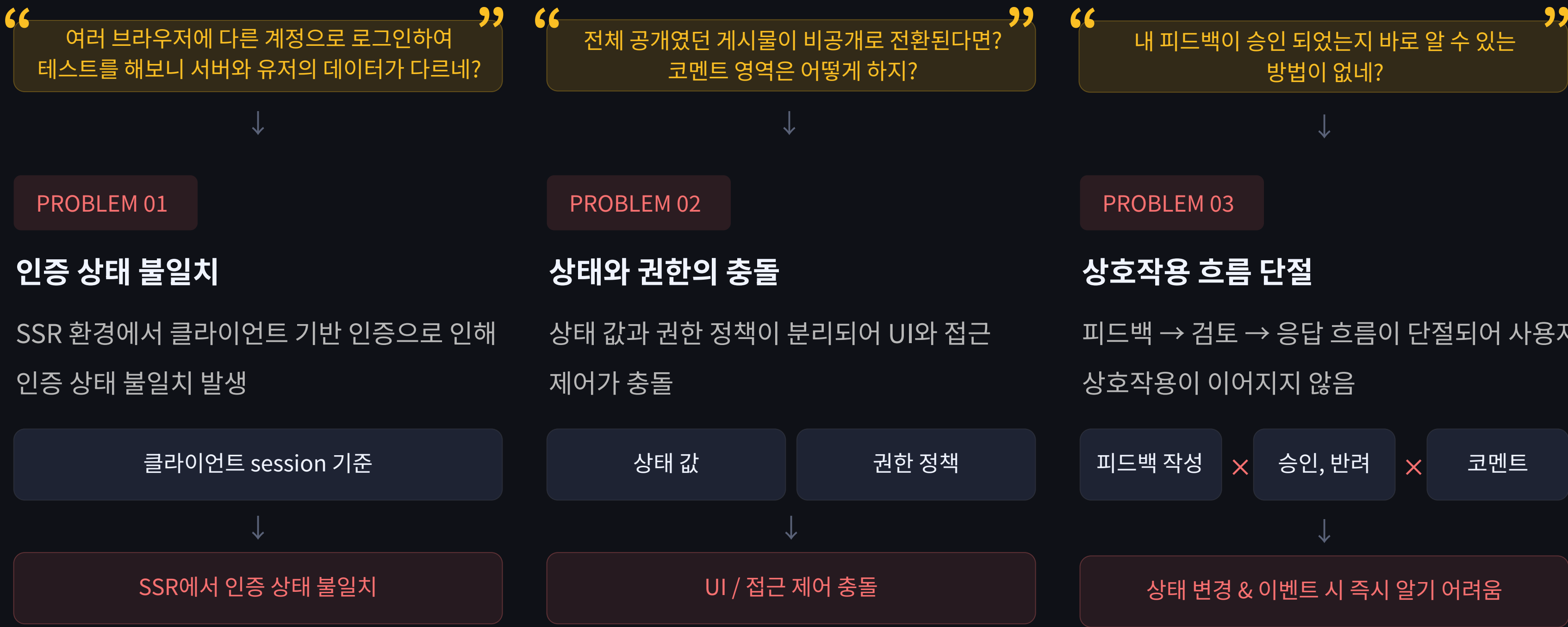
3

lint, TypeScript typecheck로 정적 검증 기준을 유지했습니다.

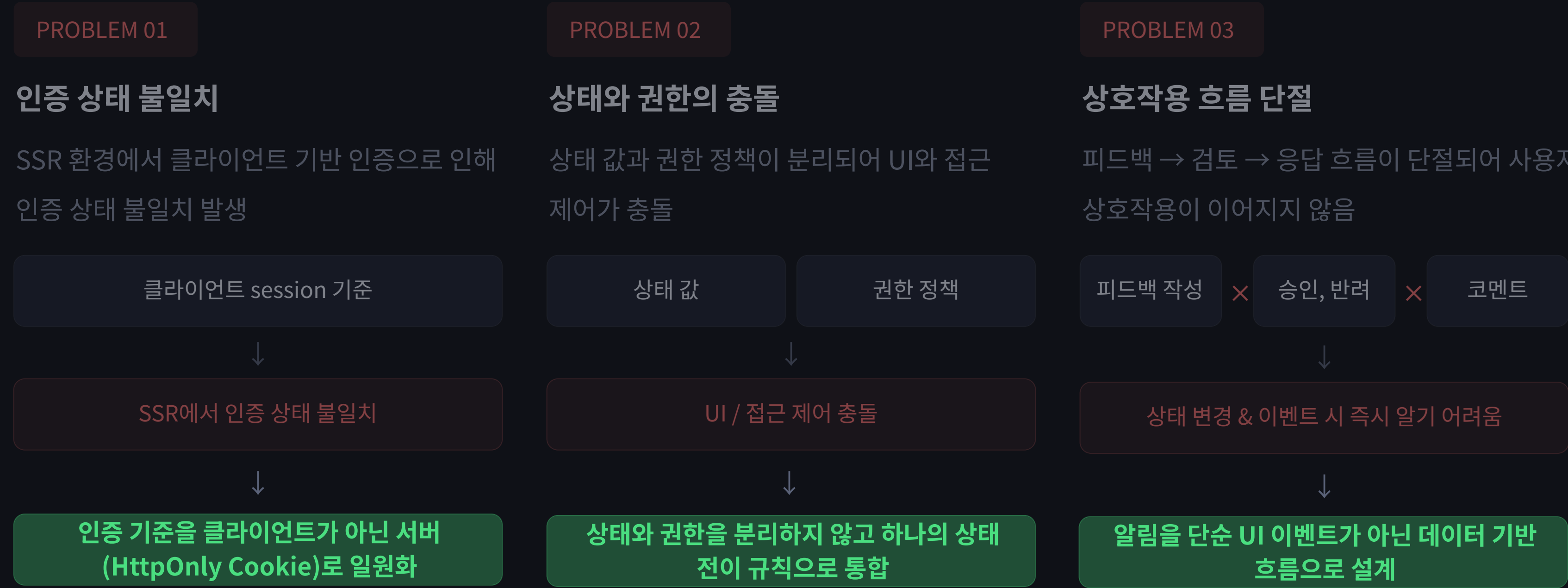
4

화면 규칙과 API 규칙이 어긋나지 않도록 주요 흐름을 함께 점검했습니다.

마주한 문제 3가지



각 문제의 해결을 위한 핵심 설계 판단



1. 인증 상태 불일치의 문제 해결 방법

서버로 인증 기준을 일원화한 인증 흐름



“Next.js 를 사용하는 만큼 서버에서 데이터를 받아오고 싶었다.”

설계 의도

클라이언트 session을 인증 기준으로 사용할 경우 SSR과 API에서 동일한 사용자 상태를 보장할 수 없었습니다.

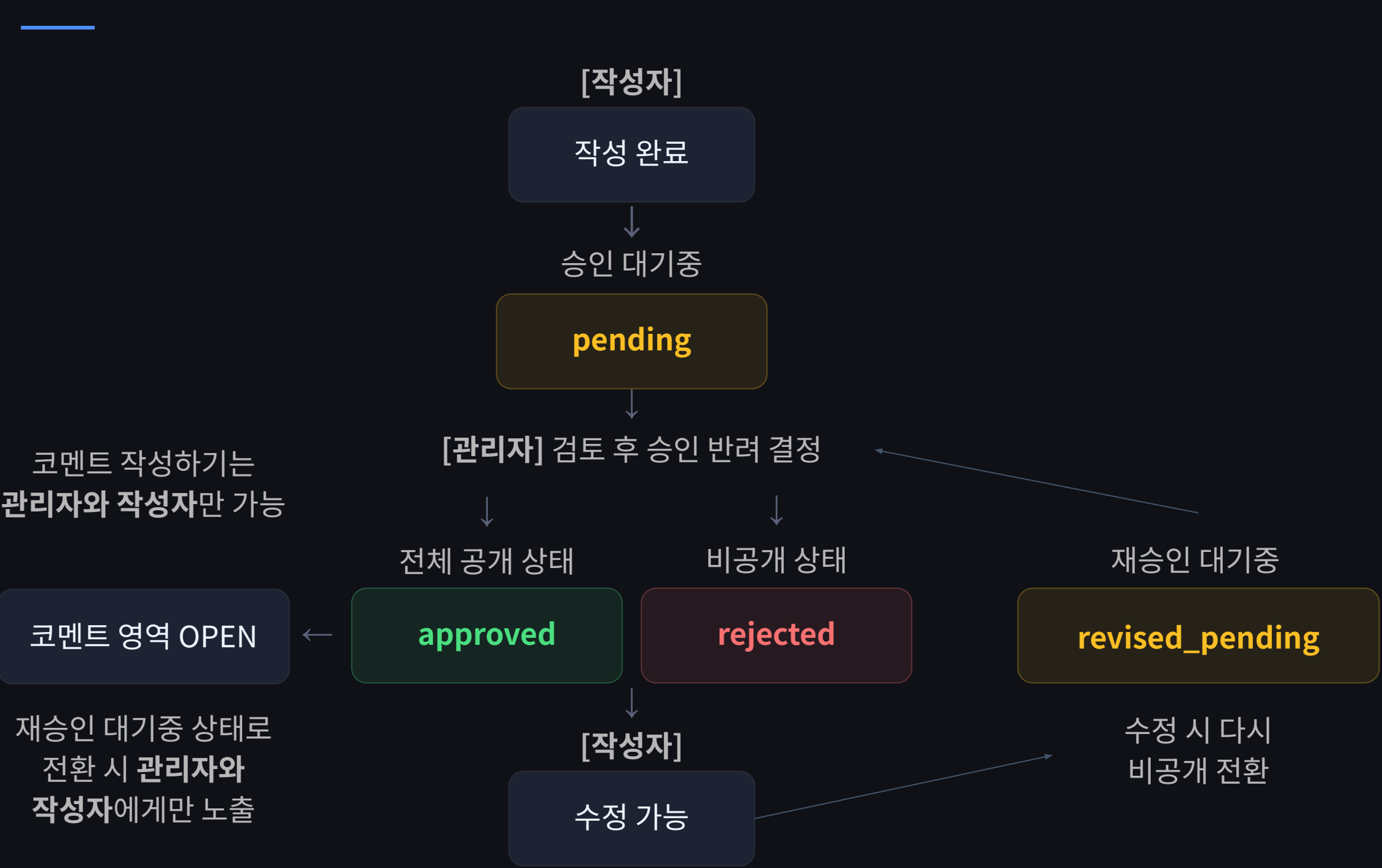
그래서 **인증 기준을 서버로 이동** 하고, 클라이언트는 UI 반응성만 담당하도록 역할을 분리했습니다.

Before
클라이언트 중심 인증
불일치 발생

After
서버 Cookie 기준
일관된 인증 검증

피드백의 상태와 사용자 권한 간 상태 전이 규칙을 통합 설계

feat. 허용 행동 & 접근 권한



pending

approved

rejected

각 상태 = 허용 행동 + 접근 권한 통합 정의

설계 의도

상태를 단순 값 변경이 아닌 **전이 규칙**으로 정의했습니다.

각 상태에서 허용되는 행동과 접근 권한을 함께 정의하여, 상태와 권한이 분리되면서 발생하던 충돌을 제거했습니다.

“프로젝트의 기능을 확장하는 과정에서, 구조를 안정적으로 유지하려면 전이 규칙을 명확히 하는 것이 핵심 원칙임을 깨달았습니다.”

[마주한 문제 1, 2번] 서버의 인증 일원화와 상태 & 권한의 규칙 정의 후 설계된

피드백 목록 페이지

관리자	작성자	비로그인 유저
재승인 대기중 revised_pending 전체 공개 상태 approved 비공개 상태 rejected 승인 대기중 pending	[작성자 본인의] 재승인 대기중 & [다른 유저의] 재승인 대기중 프리뷰 revised_pending & [프리뷰 형태] revised_pending 전체 공개 상태 approved [작성자 본인의] 비공개 상태 rejected [작성자 본인의] 승인 대기중 pending	재승인 대기중 [프리뷰 형태] revised_pending 전체 공개 상태 approved

피드백 보드 페이지 (/feedback)

앞에서 설계한 서버의 세션 인증 일원화와 접근 권한의 정의를 통해 피드백이 보이는 목록 페이지는 **유저의 권한, 피드백의 상태, 본인이 작성한 피드백에 따라 데이터를 나눠 출력하도록 설계**하였습니다.

피드백 보드 목록 페이지 보러가기

“ 승인된 공개 데이터와 사용자별 비공개 데이터를 분리해, 비로그인 사용자·작성자·관리자에게 필요한 정보만 일관된 기준으로 노출하도록 구성했습니다. ”

상호작용(알림) 구조

“누군가 피드백을 작성했을 때, 관리자가 나의 피드백을 승인하여 전체 공개로 전환되었을 때, 나의 피드백에 코멘트가 달렸을 때

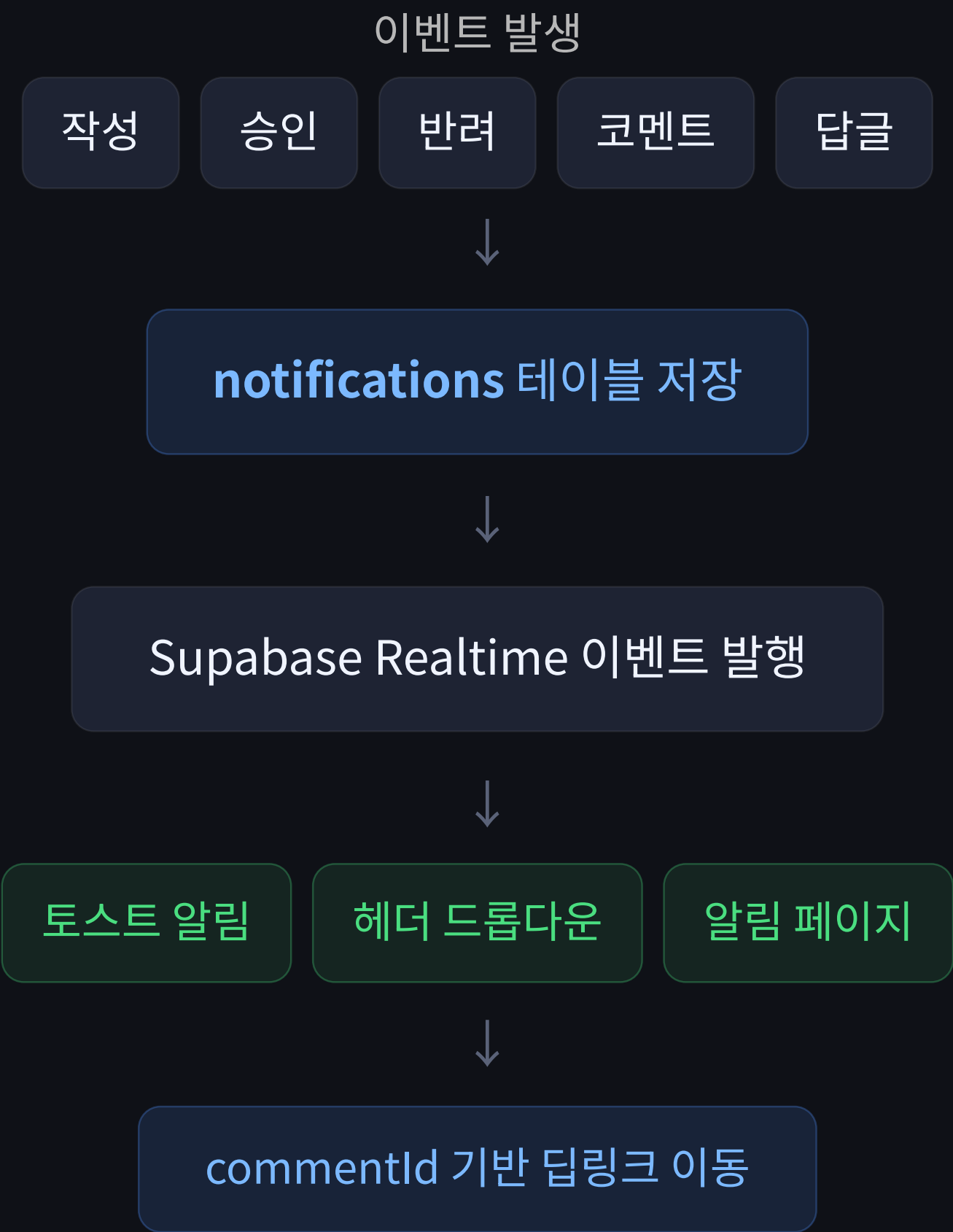
바로 알 수 있는 방법이 없어 사용자 간의 흐름과 행동이 끊기게 된다.

설계 의도

알림을 단순 UI가 아니라 "사용자 행동을 이어주는 데이터 흐름" 으로 정의했습니다.

사용자 행동 흐름

작성 → 검토 → 응답



“알림은 토스트, 헤더 드롭다운, 알림 페이지, commentId 딥링크까지 하나의 흐름으로 연결해 사용자가 다음 행동으로 바로 이어질 수 있도록 구성했습니다.

이 프로젝트의 전체 구조

(Auth 영역은 8페이지 참고)



Before / After

	Before	After
인증 기준	클라이언트 session 중심	서버 Cookie 기반
SSR 인증	불일치 발생	동일 기준 검증
상태 관리	단순 값 변경	상태 전이 규칙
권한 처리	상태와 분리	상태 + 권한 통합
상호작용	단절	알림 기반 흐름 연결

결과

- SSR / CSR / API 간 인증 기준을 통일하여 로그인 상태 불일치 가능성을 낮춤
- 상태 전이 규칙을 도입하여 권한 정책을 API 기준으로 일원화
- 알림 기반 구조를 통해 사용자 행동 흐름(작성 → 검토 → 응답 → 재방문) 연결
- Playwright 기반 E2E·API·Unit 테스트를 통해 reviewer 작성 → admin 승인 → 댓글/답글 → 알림 흐름을 검증

배운 점 & 프로젝트 기술 블로그

인사이트

- 인증은 UI 상태가 아니라 서버 기준으로 설계해야 한다
- 상태와 권한은 분리하면 복잡도가 증가한다
- 사용자 경험은 기능이 아니라 흐름으로 설계해야 한다

GitHub
[레파지토리](#)

배포 사이트
[최찬식의 인터뷰어 피드백 보드](#)

- [인터뷰어 피드백 보드 프로젝트를 시작한 이유](#)
- [아키텍처와 인증/인가 흐름](#)
- [피드백 상태 전이와 관리자 검토 워크플로우 구현](#)
- [service role과 anon key, 정말 적재적소에 쓰고 있을까?](#)
- [승인 이력 기반 코멘트 기능 설계하기](#)
- [상태 변화와 코멘트 흐름을 놓치지 않게 알림 기능 설계하기](#)
- [리팩터링 - usehooks-ts, zod, @hookform/resolvers 라이브러리 도입](#)

THANK YOU

감사합니다

퍼블리셔 최찬식

Phone

010-2385-3987

Email

ccsik0828@gmail.com

Git Hub

[CHANSIK-CHOI](#)

Blog

[Velog](#)